PRD: WhatsApp Business Broadcast & Follow-up Web App

Version: 3.0 (Draft, hosted on Supabase + Render)
Date: November 10, 2026
Owner: Product / Founder
Status: For review

Build one hosted web app in a single pass: a React frontend on Render talks to a Node backend on Render, which stores data in Supabase and sends through Meta's WhatsApp Cloud API directly. Delivery and reply updates now work without any relay, because the backend has a public address.

Free tiers are enough for a pilot, but Render's free server sleeps after 15 minutes idle and Supabase's free database pauses after a week idle, so a paying client needs about $32 a month (Render Starter $7 plus Supabase Pro $25) on top of Meta's per-message fee.

Your steps, start to finish

Do these in order; the build itself is one pass, and everything WhatsApp-related is done by the client's account setup plus pasting four values into the app.

A. Create the accounts (day 1)

1. GitHub: one private repository for the project.
2. Supabase: new project in the Mumbai region (closest to India). Save the database password, Project URL, anon key and service-role key.
3. Render: sign up and connect the GitHub account.
4. Meta: go to developers.facebook.com, create an app of type Business, and add the WhatsApp product. Note the App ID and App Secret (App settings, Basic).

B. Make the client's WhatsApp Business profile

- Create the client's Meta Business portfolio under the client's own business name, with you added as an admin or partner so you can work in it.
- Start business verification (Business Settings, Security Center) with the client's registration documents; unverified accounts have lower sending limits.
- Add a phone number that is not on any WhatsApp app, verify it by OTP, and submit the display name, which must match the business name.
- Fill the business profile in WhatsApp Manager: logo, description, address, email, website, category.

- Add a payment method in INR in WhatsApp Manager billing; Meta bills per delivered template message.
- Create a System User (Business Settings, Users, System users), give it control of the WhatsApp account and the app, and generate a token with the `whatsapp_business_messaging` and `whatsapp_business_management` permissions that never expires.
- Write down the Phone Number ID and WhatsApp Business Account ID (WhatsApp Manager, API setup).
- Create the message template (image header, text with variables) in WhatsApp Manager, or later from the app, and wait until it shows Approved.

C. Build

- Create a new project in Antigravity, paste the technical specification and the instruction text from the last section, and let it build everything.
- Run the Supabase migrations, fill the `.env` file, run locally, and test with Meta's free test number by sending to your own phone before touching the client's number.

D. Deploy and connect

- Push to GitHub; in Render create the backend Web Service and the frontend Static Site (a `render.yaml` Blueprint is specified below) and add the environment variables.
- In the Meta app, set the webhook callback to `https://<your-backend>.onrender.com/webhooks/whatsapp`, enter your verify token, and subscribe to the `messages` field.
- Set up the keep-alive and scheduler ping from the Deployment section.
- In Supabase Auth create the client's login; sign in, open Settings, and enter Phone Number ID, WhatsApp Business Account ID and the token (stored encrypted).
- Go live carefully: test message to your own number, confirm delivered and read statuses appear, pilot with about 20 opted-in contacts, then the full list while watching the number's quality rating.

Architecture and stack

The browser talks only to your backend; the backend alone talks to Supabase and to Meta. The frontend uses Supabase only to sign in, then sends the login token to the backend with every request. The WhatsApp token never reaches the browser.

Part	Choice	Runs on
Frontend	React, Vite, TypeScript, Tailwind CSS, React Router, TanStack Query and Table, React Hook Form with zod, `supabase-js` for login only	Render Static Site (free, never sleeps)
Backend API and workers	Node 20+, TypeScript, Express, zod, pino logging with masked numbers, helmet, express-rate-limit, multer for file uploads	Render Web Service
Database	Supabase Postgres; SQL migrations in `supabase/migrations`; backend connects with `pg` through Supabase's connection pooler	Supabase
Login	Supabase Auth with email and password; backend verifies the JWT on every request	Supabase
Pictures	Supabase Storage, private bucket `campaign-images`	Supabase
Queue and scheduler	Postgres tables with `FOR UPDATE SKIP LOCKED`; a worker loop inside the backend, also triggered by a `/internal/tick` endpoint	Inside the backend
Excel and CSV	SheetJS (`xlsx`) and PapaParse; upload limit 15 MB and 50,000 rows	Backend
Phone numbers	libphonenumber-js, default region IN	Backend
WhatsApp	Meta WhatsApp Cloud API over HTTPS, webhook received by the backend	Meta
Secrets	Render environment variables; each client's WhatsApp token encrypted with AES-256-GCM before it is stored	Render and Supabase
Tests	Vitest and Supertest for the backend, Playwright for end-to-end flows	CI or local

Many clients, one deployment. Every business is an organisation. Every table carries `org_id`, every query is filtered by the organisation found from the login token, and each organisation stores its own WhatsApp credentials. Roles are `owner` and `member` per

organisation, plus a `platform_admin` flag for you, which can create organisations and users. If you only ever serve one client, the same design works with a single organisation.

Request paths.

- Screen action: browser, then `POST /api/...` with `Authorization: Bearer <Supabase JWT>`, then Postgres, then response.
- Send: backend queue worker, then Meta Cloud API, then message ID saved.
- Delivery and replies: Meta, then `POST /webhooks/whatsapp`, then Postgres, then visible on the next screen refresh.

Cost and keep-alive

Fixed hosting is $0 for a pilot and about $32 a month for a client you must not let down; the per-message Meta fee is the same either way.

Item	Pilot	Production
Render frontend (Static Site)	$0	$0
Render backend (Web Service)	$0, sleeps after 15 minutes idle and may restart at any time (Render)	From $7/month for an instance that never spins down (Render pricing)
Supabase	$0: 500 MB database, 1 GB files, paused after 1 week idle, no backups (Supabase)	$25/month Pro: 8 GB database, daily backups, never paused
Scheduler and keep-alive ping	$0	$0
Domain	Not needed: Render gives you an `onrender.com` address with HTTPS	Optional
Fixed total	**$0**	**About $32/month**

Meta's fee per delivered template message to Indian numbers is about ₹0.8631 for marketing and about ₹0.115 for utility, before 18% GST (Monty Mobile). For 1,000 / 5,000 / 10,000 marketing messages that is about ₹1,018 / ₹5,092 / ₹10,185 including GST. Each follow-up is another billed message. Confirm against Meta's INR rate card before quoting.

How the pinging works, and where it falls short. Render puts a free web service to sleep after 15 minutes with no incoming traffic, and the next request waits about a minute. A request every 5 to 10 minutes keeps it awake, and one always-on free service uses about 720 of the 750 free hours in a month, so it fits only if it is your single free service (DEV Community). But Render may restart a free service at any time, so a ping is not a guarantee. The app is therefore built so a restart loses nothing:

- Every message and schedule lives in Postgres, never in memory.
- The scheduler ping calls `POST /internal/tick` every minute; the tick sends anything overdue, so a missed minute only delays work.
- Free Supabase projects pause after a week idle; the same tick touches the database every minute, which should prevent the pause. Free projects have no backups, so export the database weekly during the pilot.
- For a paying client, move the backend to the $7 instance and Supabase to Pro. This is the one upgrade that makes the keep-alive question disappear.

Use a free external scheduler such as cron-job.org, or Supabase's own `pg_cron` with `pg_net` calling the tick URL. One source reports UptimeRobot's free plan no longer allows commercial use, so check its terms before using it for a client.

Database (Supabase)

Postgres, created by numbered SQL files in `supabase/migrations`. Times are `timestamptz`. Row Level Security is switched on for every table with no policies, so nobody can read data straight from the browser with the public key; only the backend, connecting with the database connection string, can.

```
create table organizations (id uuid primary key default gen_random_uuid(),
name text not null, created_at timestamptz default now());
create table profiles (id uuid primary key references auth.users(id) on
delete cascade, full_name text, platform_admin boolean not null default
false);
create table memberships (org_id uuid references organizations(id) on delete
cascade, user_id uuid references auth.users(id) on delete cascade,
  role text not null check (role in ('owner','member')), primary key (org_id,
user_id));

create table whatsapp_accounts (
  org_id uuid primary key references organizations(id) on delete cascade,
  phone_number_id text not null, waba_id text not null,
  token_encrypted text not null,                  -- AES-256-GCM, key in env
ENCRYPTION_KEY
  app_secret_encrypted text,                      -- only if this client uses
```

```
its own Meta app
  graph_version text not null default 'v21.0',
  display_phone text, verified_name text, quality_rating text, updated_at
timestamptz default now());
create unique index on whatsapp_accounts (phone_number_id);

create table settings (org_id uuid primary key references organizations(id)
on delete cascade,
  default_country text default 'IN', timezone text default 'Asia/Kolkata',
  send_rate_per_sec int default 20, daily_send_cap int default 250,
weekly_cap_per_contact int default 3,
  rate_marketing numeric default 0.8631, rate_utility numeric default 0.115,
gst_percent numeric default 18);

create table contacts (
  id bigint generated always as identity primary key, org_id uuid not null
references organizations(id) on delete cascade,
  name text not null, whatsapp_e164 text not null, phone_raw text, email
text, extra jsonb,
  status text not null default 'active' check (status in
('active','opted_out','invalid','blocked')),
  opt_in_confirmed_at timestamptz, opt_in_source text, opted_out_at
timestamptz,
  created_at timestamptz default now(), updated_at timestamptz default now(),
  unique (org_id, whatsapp_e164));
create table lists (id bigint generated always as identity primary key,
org_id uuid not null references organizations(id) on delete cascade, name
text not null, created_at timestamptz default now());
create table list_members (list_id bigint references lists(id) on delete
cascade, contact_id bigint references contacts(id) on delete cascade, primary
key (list_id, contact_id));
create table imports (id bigint generated always as identity primary key,
org_id uuid not null, filename text, total int, valid int, invalid int,
duplicate int, created_at timestamptz default now());
create table import_errors (id bigint generated always as identity primary
key, import_id bigint references imports(id) on delete cascade, row_number
int, reason text, raw jsonb);

create table templates (
  id bigint generated always as identity primary key, org_id uuid not null
references organizations(id) on delete cascade,
  name text not null, language text not null default 'en',
  category text not null check (category in ('marketing','utility')),
  header_type text not null check (header_type in ('image','none')),
  body text not null, variable_names text[] not null default '{}', footer
text,
```

```
  meta_template_id text, status text not null default 'draft' check (status
in ('draft','pending','approved','rejected','paused')),
  rejection_reason text, created_at timestamptz default now(), updated_at
timestamptz default now(),
  unique (org_id, name, language));

create table campaigns (
  id bigint generated always as identity primary key, org_id uuid not null
references organizations(id) on delete cascade,
  name text not null, kind text not null check (kind in
('broadcast','daily','followup')),
  template_id bigint not null references templates(id), list_id bigint not
null references lists(id),
  image_path text, media_id text, media_uploaded_at timestamptz,   --
image_path = Supabase Storage path
  variables jsonb, status text not null default 'draft'
    check (status in
('draft','scheduled','running','paused','completed','cancelled','failed')),
  pause_reason text, scheduled_at timestamptz, started_at timestamptz,
completed_at timestamptz,
  total int default 0, created_by uuid, created_at timestamptz default
now());
create table daily_entries (
  id bigint generated always as identity primary key, org_id uuid not null
references organizations(id) on delete cascade,
  send_date date not null, send_time time not null, template_id bigint not
null, list_id bigint not null,
  image_path text, variables jsonb, campaign_id bigint,
  status text not null default 'planned' check (status in
('planned','sent','missed','skipped')),
  unique (org_id, send_date, template_id, list_id));

create table messages (
  id bigint generated always as identity primary key, org_id uuid not null,
campaign_id bigint not null references campaigns(id) on delete cascade,
  contact_id bigint not null references contacts(id), step_id bigint,
  idempotency_key text not null unique,            --
campaign_id:contact_id:step_id
  status text not null default 'queued' check (status in
('queued','sending','sent','delivered','read','failed','skipped')),
  wa_message_id text unique, attempt_count int default 0, next_attempt_at
timestamptz default now(),
  error_code text, error_message text,
  queued_at timestamptz default now(), sent_at timestamptz, delivered_at
timestamptz, read_at timestamptz, failed_at timestamptz);
create index on messages (status, next_attempt_at);
```

```
create index on messages (campaign_id, status);

create table inbound_messages (
  id bigint generated always as identity primary key, org_id uuid not null,
contact_id bigint, from_e164 text not null,
  wa_message_id text unique, body text, media jsonb, is_stop boolean default
false, handled boolean default false, received_at timestamptz not null);

create table sequences (id bigint generated always as identity primary key,
org_id uuid not null, name text, trigger_campaign_id bigint, active boolean
default true);
create table sequence_steps (id bigint generated always as identity primary
key, sequence_id bigint references sequences(id) on delete cascade,
  step_no int not null, delay_hours int not null, template_id bigint not
null, image_path text, variables jsonb,
  condition text not null check (condition in
('always','no_reply','delivered_not_read','read_no_reply','failed')));
create table sequence_state (contact_id bigint references contacts(id) on
delete cascade, sequence_id bigint references sequences(id) on delete
cascade,
  current_step int not null default 1, next_run_at timestamptz,
  status text not null default 'active' check (status in
('active','completed','exited_reply','exited_optout','exited_manual')),
  primary key (contact_id, sequence_id));

create table webhook_events (id bigint generated always as identity primary
key, org_id uuid, received_at timestamptz default now(), payload jsonb not
null, processed_at timestamptz);
create table audit_log (id bigint generated always as identity primary key,
org_id uuid, user_id uuid, at timestamptz default now(), action text not
null, detail jsonb);

-- security: RLS on, no policies; the backend connects as a privileged role
do $$ declare t text; begin
  for t in select tablename from pg_tables where schemaname = 'public' loop
    execute format('alter table public.%I enable row level security', t);
  end loop; end $$;
```

Storage: create the private bucket `campaign-images` (limit 5 MB, JPEG and PNG only) with no public policy; the backend reads and writes it with the service-role key and never hands out public links. Delete `webhook_events` rows older than 30 days with a nightly job.

Backend

One Node service that serves the REST API, receives Meta's webhooks and runs the background worker. All state is in Postgres, so a restart loses nothing.

```
backend/src/
  index.ts  app.ts  config.ts           env checked with zod; refuses to
start if one is missing
  db/pool.ts  db/tx.ts
  middleware/ auth.ts  errors.ts  rateLimit.ts
  routes/    me  settings  contacts  imports  lists  templates  media
campaigns  daily
             sequences  replies  reports  dashboard  admin  webhooks
internal
  services/  contacts  importer  templates  campaigns  sender  scheduler
followups  events  reports  audit
  whatsapp/  client.ts (all Meta HTTP calls)  errors.ts (code mapping)
webhook.ts (verify and parse)
  lib/       crypto.ts  phone.ts  excel.ts
supabase/migrations/    frontend/    render.yaml
```

Module	Required behaviour
auth middleware	Verify the Supabase JWT with the `jose` library against the project's JWKS (or its JWT secret if the project still uses the legacy one). Load the user's memberships, pick the organisation from the `X-Org-Id` header (default: the only one), reject non-members, and attach `{ userId, orgId, role, platformAdmin }`. Every query below filters by `orgId`.
settings	Normal settings in `settings`. The WhatsApp token and optional app secret are encrypted with AES-256-GCM using `ENCRYPTION_KEY` before saving and are never returned; the API only reports whether they are set.

Module	Required behaviour
importer	Same rules for the Excel and CSV import: 15 MB and 50,000 rows maximum; header mapping suggestions; libphonenumber-js normalisation with the organisation's default country; invalid and duplicate detection inside the file and against existing contacts; one transaction; counts in `imports`, rejected rows in `import_errors`; refused unless the consent flag is true, which stamps `opt_in_confirmed_at` and `opt_in_source`; existing opted-out contacts are never re-activated.
contacts	Search, filter, edit, add to lists, opt out, delete. A contact is selectable only when `status = 'active'` and `opt_in_confirmed_at` is set.
templates	Create a draft, check placeholder count against variable names, submit to Meta (with the image-header sample described in the integration section), store `pending`; refresh status from Meta every 30 minutes, on screen open, and from the template-status webhook. Only `approved` templates can be sent.
campaigns	Create, edit while draft, start, schedule, pause, resume, cancel, retry failed. On start: read the picture from Storage and upload it to Meta once, saving `media_id` (re-upload if older than 25 days); insert one `messages` row per selectable contact with `ON CONFLICT DO NOTHING` on `idempotency_key`; set `running`. Variables support `{{name}}` and `{{first_name}}`.

Module	Required behaviour
sender worker	Runs every second inside the service. Claims due rows with `UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED LIMIT n) RETURNING *`, only from running campaigns, then calls Meta. A per-organisation token bucket limits sends per second (default 20, maximum 80). Retryable failures back off 30 s, 2 min, 10 min, 30 min and then fail; permanent failures fail at once. When the day's distinct recipients (in the organisation's timezone) reach `daily_send_cap`, the campaign pauses with reason "daily cap" and resumes the next day. A row stuck in `sending` for over 5 minutes becomes `failed` with code `INTERRUPTED` and is never resent automatically, because a duplicate message is worse than a missed one.
tick (`POST /internal/tick`)	Protected by the `X-Cron-Secret` header, takes a Postgres advisory lock so overlapping calls do nothing, and is safe to call repeatedly. Each call: starts campaigns whose `scheduled_at` is due; turns today's due `daily_entries` into campaigns; marks entries more than 2 hours late as `missed` for the user to resolve; advances follow-up sequences; sweeps stuck messages; syncs templates when 30 minutes have passed; deletes old webhook rows nightly.
webhook (`/webhooks/whatsapp`)	`GET`: answer Meta's handshake when `hub.verify_token` matches `WEBHOOK_VERIFY_TOKEN`. `POST`: read the raw body, find the organisation from `metadata.phone_number_id`, verify `X-Hub-Signature-256` (HMAC-SHA256 of the raw body with that organisation's app secret, or `META_APP_SECRET`) using a timing-safe compare, store the payload in `webhook_events`, answer 200 immediately, then process. Unknown phone numbers or bad signatures get 403 and are logged.

Module	Required behaviour
events processing	Apply status events in order only (sent, then delivered, then read; `failed` always wins) and set timestamps. For a reply, save it in `inbound_messages` and match the contact by number. If the trimmed text equals STOP, UNSUBSCRIBE, CANCEL, END or QUIT (any case), opt the contact out, skip their queued messages and exit their sequences. Any other reply exits their active sequences with `exited_reply`. Handle `message_template_status_update` by updating the template's status and reason.
follow-ups	When a campaign completes, create `sequence_state` rows for each contact in it for every active sequence triggered by that campaign, with `next_run_at` = message `sent_at` plus step 1's delay. On each tick, for due states test the step's condition against that contact's previous message (`no_reply`, `delivered_not_read`, `read_no_reply`, `failed`, `always`); if true and the contact is active and under the weekly and daily caps, queue the step's message; if false, skip the step; then move to the next step or mark `completed`.
reports	Counts and percentages per campaign and overall, failures grouped by error, estimated spend (delivered template messages times the rate for the template category, with the GST percentage), and Excel export through SheetJS.
security	helmet; CORS allows only `FRONTEND_ORIGIN`; 300 requests per minute per IP and 20 per minute on webhook and import routes; all input validated with zod; errors never leak stack traces; logs mask numbers like +9198*****10 and never include tokens; audit log for imports, sends, cancellations, credential changes and logins.

All responses are JSON: `{ "data": ... }` on success and `{ "error": { "code", "message" } }` with the right HTTP status on failure.

Method and path	Purpose
`GET /healthz`	Light check for Render; no authentication, no database work
`GET /api/me`	User, organisations, role
`GET /api/dashboard`	Counts, next scheduled send, banners (missed sends, interrupted messages, connection problem, daily cap)
`GET /api/settings`, `PUT /api/settings`	Sending and pricing settings
`PUT /api/settings/whatsapp`, `POST /api/settings/whatsapp/test`	Save Phone Number ID, account ID, token, app secret; test the connection
`GET /api/contacts`, `GET/PATCH/DELETE /api/contacts/:id`, `POST /api/contacts/:id/opt-out`, `POST /api/contacts/bulk`	Contacts with server-side paging and filters
`POST /api/imports/preview`, `POST /api/imports/run`, `GET /api/imports/:id/errors.csv`	Excel and CSV import (multipart)
`GET/POST /api/lists`, `PATCH/DELETE /api/lists/:id`, `POST/DELETE /api/lists/:id/members`	Audiences
`GET/POST /api/templates`, `POST /api/templates/:id/submit`, `POST /api/templates/sync`, `DELETE /api/templates/:id`	Templates
`POST /api/media`	Upload a picture to Storage; returns its path
`GET/POST /api/campaigns`, `GET/PATCH /api/campaigns/:id`	Campaigns
`POST /api/campaigns/:id/start`, `/schedule`, `/pause`, `/resume`, `/cancel`, `/retry-failed`	Campaign control; `start` returns the queued recipient count
`GET /api/campaigns/:id/progress`, `GET /api/campaigns/:id/messages`	Live counts (screen polls every 3 seconds) and per-contact rows
`GET /api/daily?month=YYYY-MM`, `POST /api/daily`, `PATCH/DELETE /api/daily/:id`, `POST /api/daily/:id/resolve`	Daily planner; resolve takes `send` or `skip`

Method and path	Purpose
`GET/POST /api/sequences`, `PUT/DELETE /api/sequences/:id`, `POST /api/sequences/:id/toggle`, `GET /api/sequences/:id/states`, `DELETE /api/sequences/:id/states/:contactId`	Follow-ups
`GET /api/replies`, `POST /api/replies/:id/handled`	Customer replies
`GET /api/reports/summary`, `GET /api/reports/campaigns/:id`, `GET /api/reports/export`	Reports and Excel export
`POST /api/admin/orgs`, `POST /api/admin/orgs/:id/users`	Platform admin only: create a client organisation and its login
`GET/POST /webhooks/whatsapp`	Meta handshake and events
`POST /internal/tick`	Scheduler and keep-alive, secret header required

Frontend

A React single-page app with a left sidebar (Dashboard, Contacts, Templates, Campaigns, Daily planner, Follow-ups, Replies, Reports, Settings) and a top bar showing the organisation, WhatsApp connection state and the day's remaining send allowance. It calls only the backend REST API through one typed client module that attaches the login token, retries once on a network error, and shows a friendly message when the backend is waking up (Render's cold start takes about a minute). Use TanStack Query for data and refetching, a virtualised table for big lists, toasts for results, and a confirmation dialog before any send that shows the exact recipient count. Keep every user-facing string in one file.

Screen	What it shows	What the user can do
Login	Email and password through Supabase Auth, forgot-password link	Sign in; the session token is kept by `supabase-js`

Screen	What it shows	What the user can do
Dashboard	Active contacts, messages sent today, last campaign result, next scheduled send; banners for missed daily sends, interrupted messages, connection problems, daily cap reached	Open the item a banner points at
Contacts	Searchable, filterable table with paging; contact drawer with details, message history and sequence state	Import, add, edit, add to list, mark opted out, delete, bulk actions
Import wizard	Step 1 choose file; step 2 map columns with a 20-row preview; step 3 consent checkbox, then a summary of valid, invalid and duplicate rows	Correct the mapping, download the error list, confirm
Lists	Named audiences with member counts	Create, rename, delete, edit members
Templates	Table with Pending, Approved, Rejected badges and rejection reason; form with name, category, header image on or off, body with variable chips, and a WhatsApp-style live preview	Create, submit to Meta, refresh status, duplicate
New campaign wizard	Audience, template, picture upload and variables, preview with next and previous sample contact, review (recipients, excluded opted-out contacts, estimated cost, allowance left)	Send now, Schedule, Save as draft
Campaigns	List with status; detail page with live progress bar and counts (refreshed every 3 seconds), per-contact table with status and error	Pause, resume, cancel, retry failed, export

Screen	What it shows	What the user can do
Daily planner	Month calendar; days in the next 7 with no content are flagged; side panel with template, audience, picture, variables and send time; missed sends shown for a decision	Add or edit a day, copy from another day, skip a day, Send now or Skip a missed one
Follow-ups	Sequence list; builder with trigger campaign and step cards (delay, condition, template, picture); per-contact states	Create, enable or disable, reorder steps, remove a contact
Replies	Customer replies with time and a STOP marker	Open `wa.me/<number>` in a new tab to answer by hand, mark handled
Reports	Per-campaign and overall counts with percentages, estimated spend, failures grouped by error	Filter by date, export to Excel
Settings	Connection (Phone Number ID, WhatsApp Business Account ID, token and app secret fields that are write-only, Test connection); Sending (rate, daily cap, weekly cap, default country, timezone); Pricing (rates and GST)	Save, test
Admin (platform admin only)	Organisations and their users	Create an organisation and its first login, switch organisation

Quality rules: every list has an empty state that says what to do next; long operations show progress; every error says what happened and what to try; the app works with the keyboard and on a laptop screen of 1280 pixels; the import wizard stays responsive for 50,000 rows because the server does the parsing.

WhatsApp integration points

All Meta calls live in one file, `backend/src/whatsapp/client.ts`, with one function per row below; nothing else in the app knows Meta's URLs. Base URL is `https://graph.facebook.com/{graph_version}` with the version taken from the

organisation's `whatsapp_accounts` row so it can be changed without code. Every call sends `Authorization: Bearer <decrypted token>`. Customer numbers are sent without the plus sign.

#	Function	Meta call	Used by
1	`testConnection`	`GET /{phone-number-id}?fields=display_phone_number,verified_name,quality_rating`	Test button; saves name and quality rating
2	`subscribeApp`	`POST /{waba-id}/subscribed_apps`	Runs when credentials are saved, so Meta sends this account's webhooks to your app
3	`uploadMedia`	`POST /{phone-number-id}/media`, multipart with `messaging_product=whatsapp`, `file`, `type`	Campaign start; returns `id` saved as `media_id`
4	`uploadTemplateSample`	`POST /{app-id}/uploads?file_length=&file_type=&file_name=` returns an upload ID; then `POST /{upload-id}` with header `Authorization: OAuth <token>`, `file_offset: 0` and the image bytes returns a handle `h`	Needed once per template that has an image header; uses `META_APP_ID`
5	`createTemplate`	`POST /{waba-id}/message_templates` (body below)	Template submit
6	`listTemplates`	`GET /{waba-id}/message_templates?fields=name,status,category,language,components,rejected_reason&limit=100`, following `paging.next`	Template sync
7	`sendTemplate`	`POST /{phone-number-id}/messages` (body below)	Sender worker
8	`sendFreeForm`	`POST /{phone-number-id}/messages` with `type` `text` or `image`	Only if the contact messaged within the last 24 hours (check `inbound_messages`)

Template creation body for point 5 (the sample handle comes from point 4):

```
{
  "name": "daily_offer",
  "language": "en",
  "category": "MARKETING",
  "components": [
    { "type": "HEADER", "format": "IMAGE", "example": { "header_handle": ["
<handle h>"] } },
    { "type": "BODY", "text": "Hi {{1}}, today's offer: {{2}}",
      "example": { "body_text": [["Rahul", "20% off today only"]] } },
    { "type": "FOOTER", "text": "Reply STOP to opt out" }
  ]
}
```

Send body for point 7 (picture by uploaded media ID, so no public image URL is needed):

```
{
  "messaging_product": "whatsapp",
  "to": "919876543210",
  "type": "template",
  "template": {
    "name": "daily_offer",
    "language": { "code": "en" },
    "components": [
      { "type": "header", "parameters": [{ "type": "image", "image": { "id":
"<media_id>" } }] },
      { "type": "body", "parameters": [
        { "type": "text", "text": "Rahul" },
        { "type": "text", "text": "20% off today only" } ] }
    ]
  }
}
```

The success response holds the message ID at `messages[0].id`; save it as `wa_message_id`.

Incoming webhook (`POST /webhooks/whatsapp`), read from `entry[].changes[]`:

- `field: "messages"` with `value.statuses[]`: each has `id`, `status` (`sent`, `delivered`, `read`, `failed`), `timestamp`, `recipient_id` and, on failure, `errors[]` with `code` and `title`.
- `field: "messages"` with `value.messages[]`: a customer reply with `from`, `id`, `timestamp`, `type` and `text.body`; `value.metadata.phone_number_id` identifies the organisation.
- `field: "message_template_status_update"`: `value.event` (for example approved or rejected), `message_template_id`, `message_template_name`, `reason`. Subscribe the Meta app to this field as well as `messages`.

Error mapping in `sendTemplate` (confirm each code against Meta's current error-code reference):

- Retryable: network failures, HTTP 5xx, and rate-limit codes such as 130429 and 131056.
- Stop the whole campaign and show a banner: 190 (token expired or invalid) and 131048 (spam rate limit reached).
- Permanent, mark that message failed: 131026 (undeliverable or not on WhatsApp), 132000, 132001 and 132012 (template or parameter problems), 131049 (Meta chose not to deliver a marketing message).
- Unknown codes: permanent, with Meta's message stored in `error_message`.

Where each credential goes: Phone Number ID, WhatsApp Business Account ID and token are entered by the client or you on the Settings screen and stored per organisation (token encrypted). `META_APP_ID`, `META_APP_SECRET`, `WEBHOOK_VERIFY_TOKEN` and `ENCRYPTION_KEY` are Render environment variables.

Test before the client's number: Meta gives a free test number and lets you send to a few verified recipient numbers, which is enough to prove points 1, 3, 7 and the webhook before any real list is used.

Deployment on Render

Two services from one GitHub repository, described in a `render.yaml` at the repo root (confirm field names against Render's Blueprint reference when you create it). Do not use Render's own free Postgres; data lives in Supabase.

```
services:
  - type: web
    name: wa-backend
    runtime: node
    rootDir: backend
    plan: free                 # change to starter for a paying client
    buildCommand: npm ci && npm run build
    startCommand: node dist/index.js
    healthCheckPath: /healthz
    envVars:
      - { key: NODE_VERSION, value: "20" }
      - { key: NODE_ENV, value: production }
      - { key: DATABASE_URL, sync: false }
      - { key: SUPABASE_URL, sync: false }
      - { key: SUPABASE_SERVICE_ROLE_KEY, sync: false }
      - { key: SUPABASE_JWKS_URL, sync: false }
      - { key: ENCRYPTION_KEY, sync: false }
      - { key: META_APP_ID, sync: false }
      - { key: META_APP_SECRET, sync: false }
      - { key: WEBHOOK_VERIFY_TOKEN, sync: false }
      - { key: CRON_SECRET, sync: false }
      - { key: FRONTEND_ORIGIN, sync: false }
  - type: web
    name: wa-frontend
    runtime: static
    rootDir: frontend
    buildCommand: npm ci && npm run build
    staticPublishPath: dist
    routes:
      - { type: rewrite, source: /*, destination: /index.html }
```

```
    envVars:
      - { key: VITE_API_URL, sync: false }
      - { key: VITE_SUPABASE_URL, sync: false }
      - { key: VITE_SUPABASE_ANON_KEY, sync: false }
```

Variable	Where to get it
`DATABASE_URL`	Supabase, Project Settings, Database, connection string (use the pooler string)
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Supabase, Project Settings, API; keep the service-role key secret, backend only
`SUPABASE_JWKS_URL`	Supabase auth keys endpoint for the project (or give the JWT secret instead if the project still uses the legacy one)
`ENCRYPTION_KEY`	32 random bytes in base64, generated once; losing it makes saved WhatsApp tokens unreadable, so keep a copy somewhere safe
`META_APP_ID`, `META_APP_SECRET`	Meta app, App settings, Basic
`WEBHOOK_VERIFY_TOKEN`, `CRON_SECRET`	Any long random strings you make up; the first goes into Meta's webhook form, the second into the scheduler
`FRONTEND_ORIGIN`	The frontend's Render address, for example `https://wa-frontend.onrender.com`
`VITE_API_URL`	The backend's Render address
`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`	Supabase, Project Settings, API (the anon key is meant to be public)

Scheduler and keep-alive. Call the tick every minute from either of these, with a timeout of at least 60 seconds because the first call after a sleep waits for the server to wake:

1. cron-job.org (free): a job that sends `POST https://wa-backend.onrender.com/internal/tick` with the header `X-Cron-Secret: <CRON_SECRET>` every minute (every 5 minutes if the free plan limits it).
2. Supabase `pg_cron`, with no outside service:

```
create extension if not exists pg_cron;
create extension if not exists pg_net;
select cron.schedule('wa-tick', '* * * * *', $$
  select net.http_post(
    url := 'https://wa-backend.onrender.com/internal/tick',
    headers := '{"X-Cron-Secret": "<CRON_SECRET>"}'::jsonb,
    timeout_milliseconds := 60000);
$$);
```

Set Render's health check to `/healthz`. After the first deploy, open the backend address followed by `/healthz` to confirm it answers, then complete the Meta webhook step.

Build order and agent instructions

The agent builds everything, including the real WhatsApp calls, in one pass. The order below only keeps the work coherent; there is no mock stage. The first run against Meta uses the free test number.

1. Repository scaffold with `backend`, `frontend`, `supabase/migrations`, `render.yaml`, `.env.example` files, strict TypeScript, lint and test setup.
2. Migrations from the Database section, config loading, crypto helpers, auth middleware.
3. Settings, contacts, lists, importer and the import wizard.
4. `whatsapp/client.ts` with all eight functions, error mapping, template service and screens.
5. Campaign service, sender worker, campaign wizard and campaign detail with live progress.
6. Webhook route with signature check, event processing, replies and STOP handling.
7. Tick endpoint, daily planner, scheduled campaigns, missed-send handling.
8. Follow-up engine and builder.
9. Reports, Excel export, dashboard, admin screen.
10. Tests, README with setup and deploy steps, `render.yaml` check.

Quality rules

- Tests: importer (bad numbers, duplicates, 50,000 rows), sender worker (backoff, daily cap, no double send under two workers, stuck-row sweep), webhook (bad signature rejected, status ordering, STOP, duplicate delivery of the same event), every follow-up condition, and tenant isolation (one organisation can never read another's data).
- A mocked Meta server (for example `msw`) is allowed in tests only; production code calls the real API.

- Import of 10,000 rows finishes in under 30 seconds; the worker sustains the configured send rate.
- No secret ever appears in a log, a response or the frontend bundle.
- Lint, type check and tests pass before the build is called done.

Acceptance criteria

- A client signs in, imports a 5,000-row Excel file and sees correct valid, invalid and duplicate counts.
- An approved image template reaches every opted-in contact; delivered and read statuses appear without refreshing the database by hand.
- Replying STOP stops all further messages to that contact, including queued follow-ups.
- Killing and restarting the backend during a send never sends the same contact twice and loses no queued message.
- A daily-planner entry sends at its time within 2 minutes when the scheduler ping is running.
- Token and app secret cannot be read from any API response.

Text to paste into Antigravity (attach this whole document with it):

```
Build the complete web application described in the attached document, in one
pass.

Stack: React + Vite + TypeScript frontend (Render Static Site); Node 20 +
TypeScript +
Express backend (Render Web Service); Supabase Postgres, Auth and Storage.
The frontend calls only the backend REST API; it uses supabase-js for login
only.

Deliver:
1. The folder structure, migrations, render.yaml and .env.example files
exactly as specified.
2. Every backend module, route and background behaviour in the Backend
section, including
   the sender worker, /internal/tick, webhook handling with signature
verification,
   STOP handling, follow-up engine, and the INTERRUPTED stuck-message rule.
3. Every screen in the Frontend section.
4. The real WhatsApp Cloud API integration in backend/src/whatsapp/client.ts,
one function
   per row of the integration table, using the request bodies in the
document.
5. Tests listed under Quality rules, all passing. Mock Meta only inside
```

```
tests.
6. A README: local setup, Supabase migration steps, Render deploy steps, Meta
webhook
   setup, and how to set the scheduler ping.

Rules:
- Every query is scoped by org_id from the verified login token; RLS stays
enabled on all tables.
- Encrypt WhatsApp tokens with AES-256-GCM using ENCRYPTION_KEY; never return
or log them.
- Mask phone numbers in logs.
- Do not invent Meta behaviour: where the document says to confirm an error
code or a
  field name, make it a constant in one file and add a TODO comment.
- Work through the build order and tell me which step you are on.
```

Sources

Checked November 10, 2026. Free-tier rules and rupee rates change; confirm them before quoting a client. Only the Meta page was opened in full; the others were read as search excerpts.

- Meta: Pricing on the WhatsApp Business Platform: per-message billing, free in-window messages, INR billing deadline (opened in full).
- Render: Deploy for Free: 15-minute spin-down, 750 free hours, restarts at any time, ephemeral filesystem.
- Render pricing 2026: paid web service from $7 a month, free instances keep sleeping on any workspace plan.
- DEV Community: uptime monitoring and Render free tier: the 720-of-750-hours arithmetic and the UptimeRobot commercial-use note.
- Supabase pricing: free plan 500 MB database, 1 GB files, pause after one week idle; Pro $25 a month.
- Monty Mobile: WhatsApp API pricing in India: about ₹0.8631 marketing and ₹0.115 utility per message.